

Cosine Similarity, Worked Out by Hand

The default way to measure how alike two embeddings are — the cosine of the angle between them — computed on small vectors, with the one case that separates it cleanly from raw dot product made explicit.

What this document does. It fixes a query embedding and three document embeddings in three dimensions, and computes cosine similarity to each by hand: dot product over the product of norms. It shows why cosine ignores vector *length* and reads only *direction*, proves the bound $[-1, 1]$, and uses a document that is literally the query “said twice as loud” to expose the difference between cosine and dot product. Numbers from Appendix A.

Companion PDFs: dot product (#6), Euclidean distance (#7), and L2 normalisation (#8) use the *same* four vectors — read them together.

1 The setup

An embedding model maps text to a vector; similar meanings land in similar directions. Take a query and three candidate documents (3-D so we can compute by hand):

$$\mathbf{q} = (2, 1, 2), \quad \mathbf{a} = (1, 0, 2), \quad \mathbf{b} = (0, 3, 1), \quad \mathbf{c} = (4, 2, 4).$$

Note $\mathbf{c} = 2\mathbf{q}$ exactly — think of it as the same content as the query but “written at twice the volume.” Keep an eye on it.

Intuition

Cosine asks one question: *do these two vectors point the same way?* It throws away how *long* each vector is and keeps only the angle between them. That is exactly what you want for text: a one-line note and a ten-page essay on the same topic point in nearly the same direction, even though the essay’s vector is far longer. Cosine calls them similar; a length-sensitive metric might not.

2 The formula

For vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$,

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_i u_i v_i}{\sqrt{\sum_i u_i^2} \sqrt{\sum_i v_i^2}}$$

The numerator is the dot product (how much the vectors co-vary); the denominator divides out both magnitudes, leaving a pure number in $[-1, 1]$: 1 = same direction, 0 = orthogonal (unrelated), -1 = opposite.

3 Worked by hand

First the norms (lengths): $\|\mathbf{q}\| = \sqrt{4 + 1 + 4} = \sqrt{9} = 3$, $\|\mathbf{a}\| = \sqrt{1 + 0 + 4} = \sqrt{5} = 2.2361$, $\|\mathbf{b}\| = \sqrt{0 + 9 + 1} = \sqrt{10} = 3.1623$, $\|\mathbf{c}\| = \sqrt{16 + 4 + 16} = \sqrt{36} = 6$.

Query vs. document a.

$$\mathbf{q} \cdot \mathbf{a} = 2 \cdot 1 + 1 \cdot 0 + 2 \cdot 2 = 6, \quad \cos = \frac{6}{3 \times 2.2361} = \frac{6}{6.7082} = 0.8944.$$

Query vs. document b.

$$\mathbf{q} \cdot \mathbf{b} = 0 + 3 + 2 = 5, \quad \cos = \frac{5}{3 \times 3.1623} = \frac{5}{9.4868} = 0.5270.$$

Query vs. document c (= 2q).

$$\mathbf{q} \cdot \mathbf{c} = 8 + 2 + 8 = 18, \quad \cos = \frac{18}{3 \times 6} = \frac{18}{18} = 1.0000.$$

Ranking by cosine: $\mathbf{c}$ (1.000) $>$ $\mathbf{a}$ (0.894) $>$ $\mathbf{b}$ (0.527). The query’s “louder twin” $\mathbf{c}$ scores a perfect 1 — identical direction, length irrelevant.

Mini-example · length cannot fool cosine

$\mathbf{c}$ is $\mathbf{q}$ scaled by 2. In the formula that 2 appears once on top ($\mathbf{q} \cdot 2\mathbf{q} = 2\|\mathbf{q}\|^2 = 18$) and once on the bottom ($\|2\mathbf{q}\| = 2\|\mathbf{q}\| = 6$), so it cancels:

$$\cos(\mathbf{q}, 2\mathbf{q}) = \frac{2\|\mathbf{q}\|^2}{\|\mathbf{q}\| \cdot 2\|\mathbf{q}\|} = \frac{2\|\mathbf{q}\|^2}{2\|\mathbf{q}\|^2} = 1.$$

Scaling *either* vector by any positive constant leaves cosine unchanged. (Dot product, PDF #6, would have jumped from 9 to 18.)

4 Why the bound is $[-1, 1]$

The Cauchy–Schwarz inequality states $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\| \|\mathbf{v}\|$, with equality only when the vectors are parallel. Dividing through by $\|\mathbf{u}\| \|\mathbf{v}\|$ gives $|\cos| \leq 1$. So cosine is always a valid “angle” — +1 when parallel (same direction), −1 when anti-parallel, 0 when perpendicular. For most embedding models the components are mixed-sign, so real similarities sit in roughly $[0, 1]$, and two *unrelated* passages score near 0, not −1.

Intuition

A subtle trap: cosine of 0.5270 for $\mathbf{b}$ does *not* mean “53% relevant.” Cosine is not calibrated — the gap between 0.89 and 0.99 can matter far more than the gap between 0.20 and 0.30. Use it to *rank* candidates, not as an absolute relevance percentage. The right threshold is dataset- and model-specific, and you find it empirically (Track 6), never by assuming 0.5 = “half.”

5 The link to normalisation and dot product

If both vectors are first scaled to unit length (PDF #8), their norms are 1 and the denominator vanishes:

$$\cos(\mathbf{u}, \mathbf{v}) = \hat{\mathbf{u}} \cdot \hat{\mathbf{v}}, \quad \hat{\mathbf{u}} = \mathbf{u} / \|\mathbf{u}\|.$$

On normalised vectors, cosine *is* the dot product. This is why vector databases that store unit-length embeddings can advertise “dot-product search” and “cosine search” as the same operation — and why normalising up front (once, at index time) makes every query cheaper. Check: $\hat{\mathbf{q}} \cdot \hat{\mathbf{a}} = 0.8944$, exactly the cosine above.

Equation cheat-sheet

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \in [-1, 1], \quad \cos(\mathbf{u}, \alpha \mathbf{v}) = \cos(\mathbf{u}, \mathbf{v}) \quad (\alpha > 0)$$

$$\cos(\mathbf{u}, \mathbf{v}) = \hat{\mathbf{u}} \cdot \hat{\mathbf{v}} \text{ (unit vectors),} \quad 1 = \text{same dir, } 0 = \text{orthogonal, } -1 = \text{opposite}$$

Appendix A · Reference implementation

```
import numpy as np
```

```
def cosine(u, v):
    u, v = np.asarray(u, float), np.asarray(v, float)
    return (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
```

```
q = [2,1,2]
cosine(q, [1,0,2])    # 0.8944  (doc a)
cosine(q, [0,3,1])    # 0.5270  (doc b)
cosine(q, [4,2,4])    # 1.0000  (doc c = 2q, length ignored)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0